Informe Práctico Final

Calidad de Software A

Estrategia de Testing





Vazquez Baccaro, Mateo
Acevedo Frías, Valentina
Gregorini, Dante Emilio
Gribman, Ianai

11 de noviembre del 2026

Índice

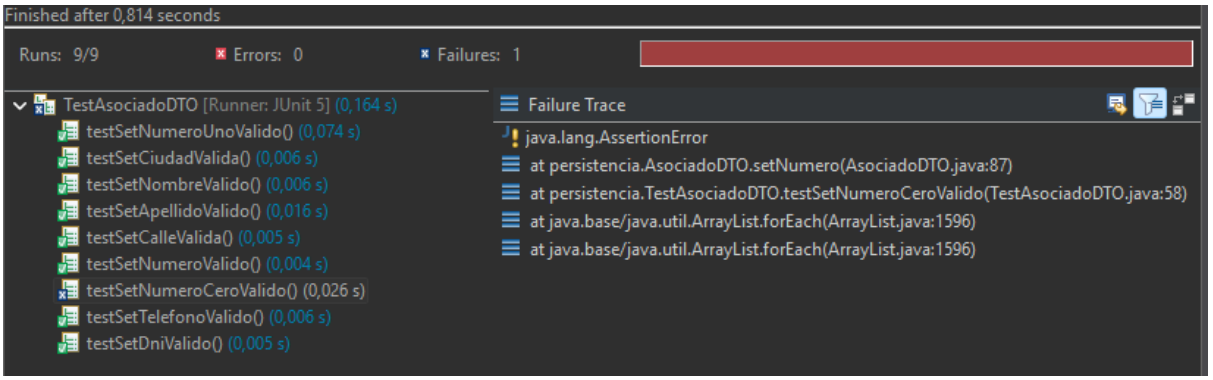
Índice **1**
Testing de la clase AsociadoDTO **2**
Testing de la integración AsociadoDTO, BaseDeDatos y DataAccessObject **3**
1. Relación entre AsociadoDTO, DataAccessObject y BaseDeDatos 3
2. Tests orientados y justificación de escenarios 3
Enfoque general de la prueba 3
Escenarios planteados 4
3. Errores encontrados en la ejecución 4
4. Errores estáticos en documentación 6
5. Nota de entorno: configuración de MySQL para las pruebas de integración 7
Testing de integración con Mocks de Ambulancia, la interfaz State y la rutina concurrente Retorno Automático **9**
1. Test de delegación 9
Escenario planteado 9
Casos de prueba 9
2. Test de transiciones de estado 10
Escenarios planteados 10
Casos de prueba 10
3. Test de concurrencia 11
Escenarios planteados 11
Casos de prueba 12
4. Errores encontrados en la ejecución 13
Resumen de fallos detectados **16**

Testing de la clase AsociadoDTO

Lo primero que notamos es que la clase `AsociadoDTO` es una clase muy simple que no cuenta con ninguna colección, por lo cual únicamente se necesita un escenario y clase de prueba para todos los métodos a testear.
Luego identificamos los métodos setter que íbamos a testear en la clase `AsociadoDTO` y que parámetros con que valores aceptaban los mismos:

1. `setNombre(String nombre)`
2. `setApellido(String apellido)`
3. `setDni(String dni)`
4. `setCiudad(String ciudad)`
5. `setCalle(String calle)`
6. `setNumero(int numero)`
7. `setTelefono(String telefono)`

En el caso de todos los métodos con parámetro string tienen como precondición que el string no es nulo, por lo cual lo único que podemos testear es que efectivamente ingresando cualquier string no nulo este se guarde, para el método `setNumero(numero)`, está como precondición que el número debe ser >= 0, por lo que testear el caso límite que el valor sea 0, y otro en el que es 1, ahí identificamos que el código tiene un assert numero > 0, lo cual sería un error ya que en el contrato incluye al 0 como un número válido



Un detalle a considerar es que utilizamos Junit 5 en lugar de Junit4, por lo que `@Before` y `@After` pasan a ser `@BeforeEach` y `@AfterEach`, pero no cambia en nada la resolución, en el caso de before se genera un `AsociadoDTO` nuevo cada vez para probar desde 0 y en el after no se necesita hacer nada

Testing de la integración AsociadoDTO, BaseDeDatos y DataAccessObject

1. Relación entre AsociadoDTO, DataAccessObject y BaseDeDatos

El módulo de persistencia implementa una arquitectura en capas orientada a separar el acceso a datos, la lógica de transferencia y la gestión de conexiones con la base de datos relacional MySQL. La interacción entre sus componentes se estructura de la siguiente manera:

- **AsociadoDTO (Data Transfer Object):** actúa como un contenedor plano de datos (Serializable) que transporta la información de la entidad Asociado entre la capa de dominio y la capa de persistencia, desacoplando los objetos de negocio de la estructura de la base de datos.
- **DataAccessObject (DAO):** Es el componente encargado de coordinar las operaciones CRUD sobre la entidad Asociado. Recibe objetos de dominio, los convierte a `AsociadoDTO` mediante métodos de mapeo y utiliza la capa inferior para ejecutar las sentencias SQL parametrizadas correspondientes.
- **BaseDeDatos:** implementa el patrón **Singleton** para garantizar una única instancia de conexión activa (Connection) hacia el motor MySQL. Provee los métodos genéricos de ejecución (`ejecutarConsulta`, `ejecutarInsert`, `ejecutarActualizacion`, `ejecutarDelete` y `iniciarBD`) utilizados directamente por el `DataAccessObject`.

2. Tests orientados y justificación de escenarios

Para validar el correcto funcionamiento del módulo, se diseñó una batería de pruebas de integración estructurada mediante clases base (`BaseIntegracion`) y una suite unificada (`SuiteIntegracionPersistencia`). La estrategia de prueba se divide enfoques generales y escenarios controlados:

Enfoque general de la prueba

- **Aislamiento por Escenarios:** se garantiza que cada prueba comience en un estado determinista preconfigurado gracias a los métodos de ciclo de vida (`@BeforeEach` y `@AfterEach`) y al reseteo del Singleton de la base de datos.
- **Independencia del DAO en Verificaciones:** para comprobar el éxito de las operaciones de escritura del `DataAccessObject`, se utilizan *helpers* de verificación basados en JDBC directo (`contarFilas`, `existeDni`, `valorColumna`) para evitar dependencias circulares con los métodos de lectura del propio DAO.

Escenarios planteados

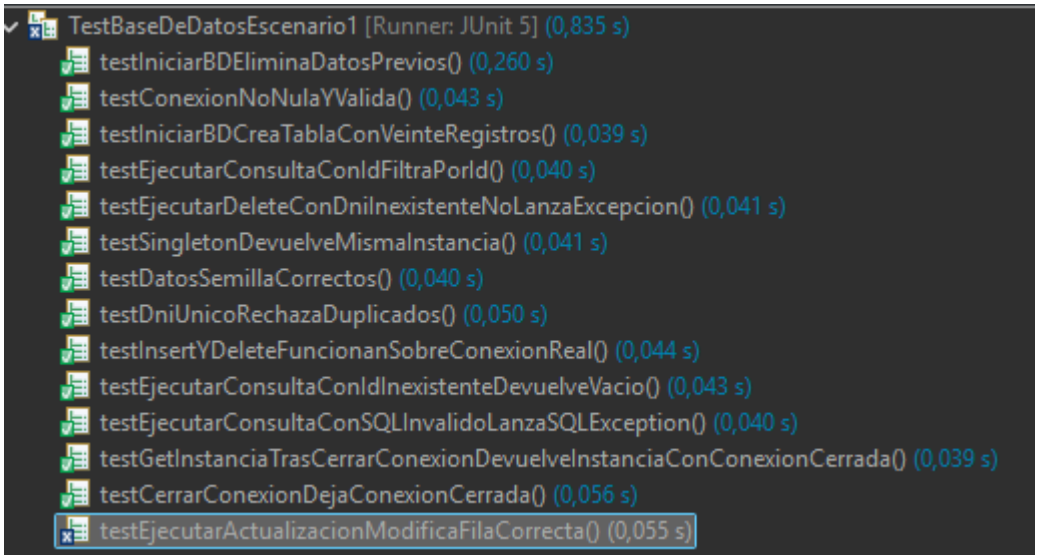
1. **Escenario 1 (Conexión abierta y 20 registros semilla):**
   - *Objetivo:* validar el comportamiento nominal del sistema bajo condiciones estándar de operación. Comprueba la correcta carga masiva, filtrados por ID, límites de consulta, inserciones válidas, restricciones de unicidad (DNI duplicado) y operaciones completas de CRUD.

2. **Escenario 2 (Conexión abierta y tabla vacía):**
   - *Objetivo:* evaluar el comportamiento ante condiciones límite de volumen de datos (tabla inicializada mediante TRUNCATE). Verifica que las consultas masivas devuelvan colecciones vacías, que las inserciones creen registros desde cero y que la reinicialización repueble correctamente los datos semilla.

3. **Escenario 3 (Conexión cerrada):**
   - *Objetivo:* comprobar la robustez y el manejo de excepciones ante fallas de conectividad. Valida que cualquier intento de ejecución de operaciones CRUD con la conexión cerrada propague de manera controlada una excepción de tipo SQLException.

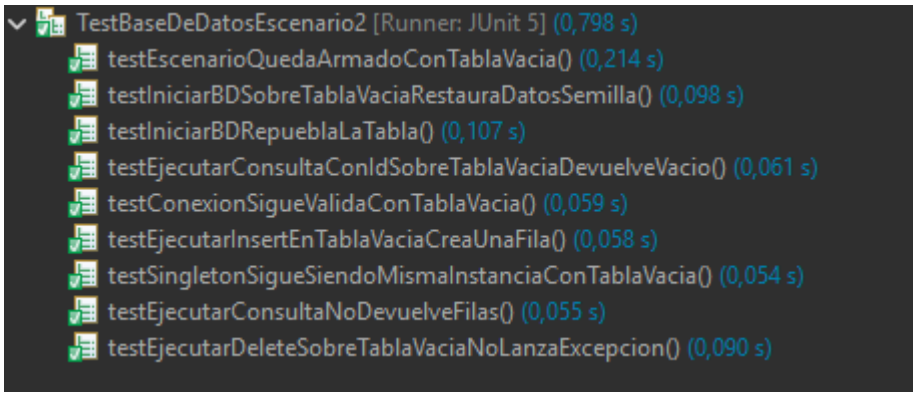
3. Errores encontrados en la ejecución

- **BaseDeDatos:**
  - **Escenario 1:**

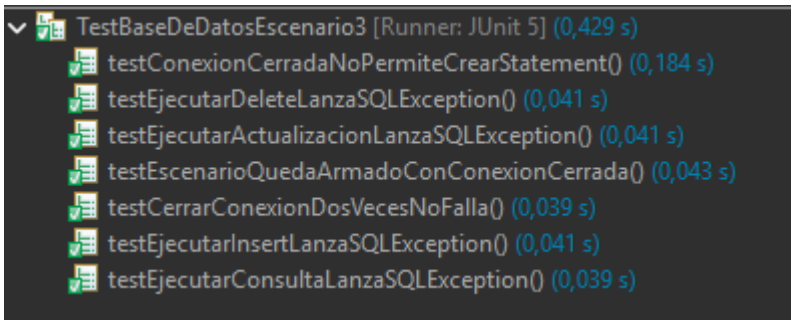


    La conexión Singleton se cumple, y sus funcionalidades manejan de manera correcta la base de datos, el único error encontrado es:
    - En el método `ejecutarActualizacion()`, el cual recibe una query SQL para ejecutar un UPDATE con parámetros de un asociadoDTO para actualizar sus datos, se espera que actualice la fila correcta, pero en el código está hardcodeado que siempre modificara la fila 1 de la tabla.

- **Escenario 2:**

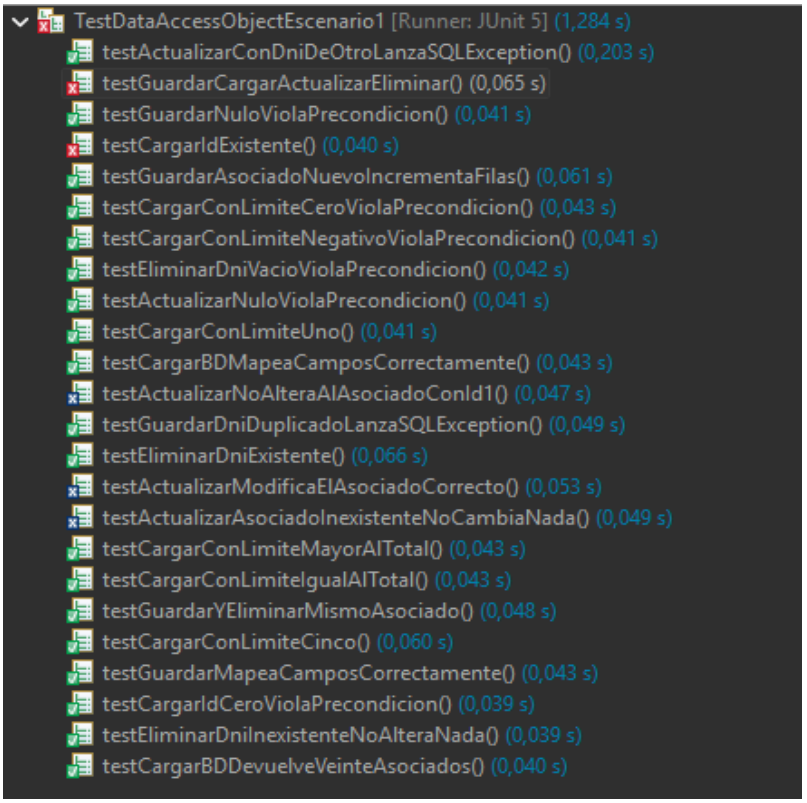


- **Escenario 3:**



Escenario 2 y 3 comprobaron que los métodos funcionan de manera correcta bajo esas condiciones (tabla vacía y sin conexión).

- **DataAccessObject:**
  - **Escenario 1:**

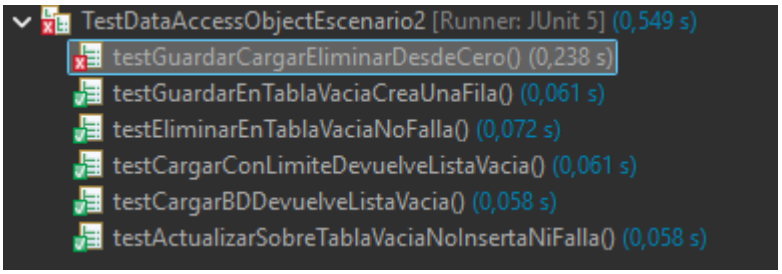


Se encontraron varias fallas:

- El método `cargar()` no funciona bajo cualquier parámetro que se le envíe, lanza excepción en todos los tests que lo ejecutan, esto es porque utiliza la clase `ResultSet` para obtener el resultado de una query enviada a la base de datos como consulta de un asociado por id, tal clase posee un cursor que apunta a una fila de la lista que contiene, inicializada en la posición anterior a la 1er fila ("no apunta a nada"), y bajo esa posicion inicial ejecutan getInt() o getString() para acceder a campos del registro, como no hay registro apuntado en ese momento lanza SQLException.
  Fue encontrado en `testGuardarCargarActualizarEliminar()`, `testCargarIdExistente()`.
- `actualizar()`, al usar el método `ejecutarActualizacion()` de la base de datos, siempre actualiza a la fila con id 1 independientemente del que se le pase como parámetro.
  Fue encontrado en `testActualizarNoAlteraAlAsociadoConId1()`, `testActualizarModificaElAsociadoCorrecto()`, `testActualizarAsociadoInexistenteNoCambiaNada()`.

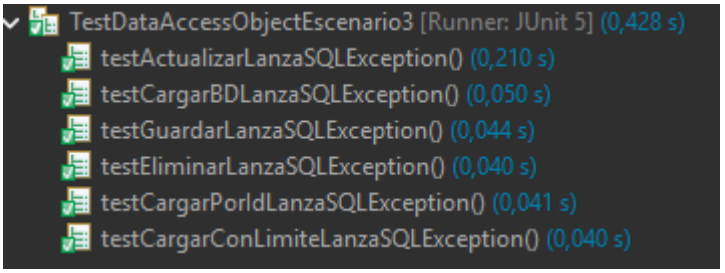
Las demás operaciones se consideran que funcionan correctamente.

- **Escenario 2:**



Se encontró la misma falla que en el anterior escenario: `cargar()` no funciona y lanza SQLException. El resto de operaciones se consideran funcionando correctamente bajo estas condiciones.

- **Escenario 3:**



No se encontraron fallas bajo estas condiciones.

4. Errores estáticos en documentación



Estos errores se consideran no propios del funcionamiento de los módulos, sino de la documentación provista, imposibilitando pruebas de algunas operaciones bajo condiciones y/o secuencias específicas.

- **Discrepancias en Datos Semilla (`BaseDeDatos`):**
  - Se documentó que el registro semilla con DNI 21842190 posee un espacio inicial en el campo ciudad (" Mar del Plata"), lo cual puede afectar comparaciones estrictas de cadenas si no se normaliza adecuadamente.
- **Contrato incoherente en el método eliminar() en la clase `DataAccessObject`:**
  - En la precondición del método aclara que el parámetro id debe ser mayor que 0, y que elimina un Asociado por su ID, pero el método únicamente cuenta con un parámetro dni que es string (realmente el método elimina por dni, no id). Además el método tiene un assert (dni != cadena vacía), pero el mensaje de error es que el dni no puede ser nulo, siendo estas 2 condiciones distintas.
- **Falta de Especificación de Contratos:**
  - **En DataAccessObject:**
    - No se especifica cómo debería responder el método `cargar()` bajo un id inexistente en la base de datos, si debería devolver null o lanzar una excepción (ya sea en una tabla vacía o llena).
    - No se especifica cómo debería comportarse si se intenta persistir un valor String que exceda la longitud máxima definida por el tipo VARCHAR en la base de datos.
    - No se especifica cómo identifica a un asociado a actualizar dentro de la tabla para modificar los campos del correcto, simplemente se pasa como parámetro un asociadoDTO (no hay id, no se especifica si toma algún campo de este como identificador dentro de la BD), tomamos como criterio que utiliza el dni (por lo tanto este campo se vuelve imposible de actualizar).
- **Inicio de conexión inadecuada:**
  - Para iniciar una conexión con la base de datos, se debe acceder al método `getInstancia()`, propio del Singleton, de `BaseDeDatos`. Este método llama al constructor de la misma solo si no existe instancia alguna, el cual inicia la conexión e inicializa la base de datos siempre (esto es, elimina la tabla existente que hubiera, crea una nueva y la llena de datos semilla). Al cerrar conexión con la BD, no existe manera de volver a abrirla, pues tales métodos que realizan esa operación solo son accesibles desde el método getInstancia(), el cual solo devolverá la instancia ya existente en vez de llamar al constructor (los métodos que inician la conexión, e inicializan la BD son privados). Por esto, no se persiste información al cerrar y abrir el programa, pues siempre se elimina y vuelve a crear la tabla. Por la imposibilidad de reabrir la conexión bajo la misma ejecución del programa es imposible probar que esto ocurre.

5. Nota de entorno: configuración de MySQL para las pruebas de integración

Durante la puesta a punto de las pruebas de integración del módulo de persistencia se identificaron dos condiciones del entorno que impidieron la conexión inicial contra el motor MySQL. Ninguna de ellas constituye un defecto del código provisto; se documentan aquí para garantizar la reproducibilidad de la suite y para distinguirlas de los hallazgos reales reportados con anterioridad.

1. Permisos insuficientes del usuario de base de datos
Síntoma: java.sql.SQLSyntaxErrorException: Access denied for user 'progra_c'@'localhost' to database 'grupo_7' (error 1044 de MySQL).

Causa: el usuario progra_c no contaba con privilegios sobre la base Grupo_7. El método iniciarBD() de BaseDeDatos ejecuta CREATE DATABASE IF NOT EXISTS Grupo_7, operación que el servidor rechaza si el usuario no tiene permiso de creación o acceso sobre ese esquema.

Solución aplicada (sobre el servidor, sin modificar el código):
Ejecutar estas instrucciones SQL:
GRANT ALL PRIVILEGES ON Grupo_7.* TO 'progra_c'@'localhost';
GRANT CREATE ON *.* TO 'progra_c'@'localhost';
FLUSH PRIVILEGES;

2. Plugin de autenticación incompatible con la URL de conexión
Síntoma: java.sql.SQLNonTransientConnectionException: Public Key Retrieval is not allowed.

Causa: MySQL 8.0+ utiliza por defecto el plugin de autenticación caching_sha2_password. Dado que la URL de conexión definida en BaseDeDatos especifica useSSL=false, el driver JDBC requiere obtener la clave pública del servidor para cifrar la contraseña, operación que por motivos de seguridad está deshabilitada salvo que se habilite explícitamente mediante el parámetro allowPublicKeyRetrieval=true. Dado que la consigna establece no modificar el código provisto, se optó por ajustar el plugin de autenticación del usuario del lado del servidor.

Solución aplicada (sobre el servidor, sin modificar el código):

ALTER USER 'progra_c'@'localhost' IDENTIFIED WITH mysql_native_password BY 'progra_c';
FLUSH PRIVILEGES;

Testing de integración con Mocks de Ambulancia, la interfaz State y la rutina concurrente Retorno Automático

1. Test de delegación

Este grupo evalúa la clase `Ambulancia` de forma aislada. El atributo `estadoActual` se reemplaza por un mock mediante `setState()` y se observa qué le pidió `Ambulancia` utilizando `verify` de Mockito. Cada test de delegación aplica dos verificaciones:

- `verify(estadoMock, times(1))` sobre el método esperado, que comprueba que fue invocado exactamente una vez. Detecta una delegación omitida o repetida.
- `verifyNoMoreInteractions(estadoMock)`, que comprueba que no se invocó ningún otro método de State. Detecta una delegación al método equivocado, aunque además se haya llamado al correcto.

Escenario planteado

Se parte de una ambulancia recién creada a la que se le asigna el estado mock con `setState(estadoMock)`. También se coloca `isSimulacionActiva` en `true` y `ocupado` en `false`.

Casos de prueba

Método	Resultado esperado
`solicitarMantenimiento(operarioMock)`	`estadoActual` recibe una única llamada de `SolicitudMantenimiento()` y ninguna otra
`atenderDomicilio(asociadoMock)`	`estadoActual` recibe una única llamada de `SolicitudDeAtencionDomicilio()` y ninguna otra
`trasladarALaClinica(asociadoMock)`	`estadoActual` recibe una única llamada `de SolicitudDeTraslado()` y ninguna otra

`retornoAutomatico(retornoAutomaticoMock)`	`estadoActual` recibe una única llamada de `RetornoClinica()` y ninguna otra

2. Test de transiciones de estado

Este grupo es un test de integración entre la clase `Ambulancia` y las implementaciones concretas de la interfaz `State`. A diferencia del grupo anterior, no se reemplaza el estado por un mock: la ambulancia y los estados son reales. El objetivo es verificar que, para cada estado inicial y cada método de la interfaz (`RetornoClinica`, `SolicitudDeAtencionDomicilio`, `SolicitudMantenimiento` y `SolicitudDeTraslado`), la ambulancia quede en el estado que indica la tabla de transiciones del enunciado.

Escenarios planteados

Todos los escenarios tienen en común que se coloca `isSimulacionActiva` en `true` y `ocupado` en `false`. Lo que diferencia cada escenario es la instancia real de uno de los seis estados concretos que se asigna a la ambulancia recién creada, mediante `setState()`.

- **Escenario 1**: estado inicial = `DisponibleState`
- **Escenario 2**: estado inicial = `TrasladandoPacienteClinicaState`
- **Escenario 3**: estado inicial = `AtendiendoEnDomicilioState`
- **Escenario 4**: estado inicial = `RegresandoClinicaSinPacienteState`
- **Escenario 5**: estado inicial = `EnTallerState`
- **Escenario 6**: estado inicial = `RegresandoDelTallerState`

Casos de prueba

La siguiente tabla resume los casos de prueba. Cada celda combina un método de la interfaz State y un escenario. El contenido de la celda representa la salida esperada, que se manifiesta mediante un cambio de estado o manteniéndose inalterado. Los métodos en determinados estados también envían mensajes a objetos ajenos al patrón State, por lo que no se probarán en este test.

Estado actual de la ambulancia	`RetornoClinica()`	`SolicitudDeAtencionDomicilio()`	`SolicitudMantenimiento()`	`SolicitudDeTraslado()`
1. Disponible	No cambia el	3	5	2



	estado			
2. Trasladando paciente	1	No cambia el estado	No cambia el estado + notifica a observers que no se permite	No cambia el estado + notifica a observers que no se permite
3. Atendiendo a domicilo	4	No cambia el estado	No cambia el estado + notifica a observers que no se permite	No cambia el estado + notifica a observers que no se permite
4. Regresando sin paciente	1	3	No cambia el estado + notifica a observers que no se permite	2
5. En taller	No cambia el estado	No cambia el estado	6	No cambia el estado + notifica a observers que no se permite
6. Regresando del taller	1	No cambia el estado	No cambia el estado + notifica a observers que no se permite	No cambia el estado + notifica a observers que no se permite

3. Test de concurrencia

Este grupo evalúa la `Ambulancia` como recurso compartido entre hilos reales, y se comprueba que un hilo espere si está ocupada y continúe al liberarse, que no espere si está libre o si la simulación finalizó, y que dos hilos no accedan a la vez. También se evalúa el ciclo de vida del hilo de `RetornoAutomatico` según el estado de la simulación.

Escenarios planteados

- **Escenario 1 (ambulancia ocupada, simulación activa):** se parte de una ambulancia recién creada a la que se le asigna el estado mock con `setState(estadoMock)` y se le coloca `isSimulacionActiva` en `true` y `ocupado` en `true`.
- **Escenario 2 (ambulancia ocupada, simulación finalizada):** se parte de una ambulancia recién creada a la que se le asigna el estado mock con `setState(estadoMock)` y se le coloca `isSimulacionActiva` en `false` y `ocupado` en `true`.
- **Escenario 3: (ambulancia libre, simulación activa):** se parte de una ambulancia recién creada a la que se le asigna el estado mock con `setState(estadoMock)` y se le coloca `isSimulacionActiva` en `true` y `ocupado` en `false`.



- **Escenario 4 (ambulancia ocupada, simulación activa, sin estado mock):** se parte de una ambulancia recién creada a la que se le coloca `isSimulacionActiva` en `true` y `ocupado` en `true`.
- **Escenario 5 (ambulancia libre, simulación activa, sin estado mock):** se parte de una ambulancia recién creada a la que se le coloca `isSimulacionActiva` en `true` y `ocupado` en `false`.
- **Escenario 6 (ambulancia libre, simulación finalizada, sin estado mock):** se parte de una ambulancia recién creada a la que se le coloca `isSimulacionActiva` en `false` y `ocupado` en `false`.

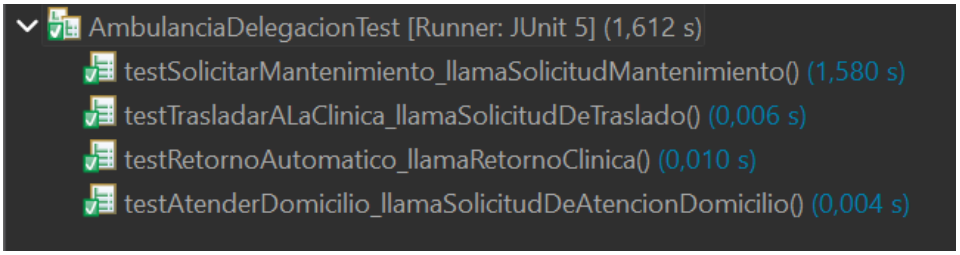
Casos de prueba

Objetivo	Escenario	Acción	Resultado esperado
Que un hilo quede esperando mientras la ambulancia está ocupada y continúe al liberarse	1	Un hilo invoca `trasladarALaClinica(asociadoMock)` y luego se libera la ambulancia	El hilo queda en `WAITING` y el estado no recibe llamadas. Al liberarla, el hilo termina y el estado recibe una única llamada a `SolicitudDeTraslado()`
Que un hilo en espera termina si la simulación finaliza, aunque la ambulancia esté ocupada	1	Se invoca `trasladarALaClinica(asociadoMock)` y luego se finaliza la simulación	El hilo se despierta y termina, y el estado no recibe llamadas
Que, con la simulación finalizada, el hilo no espere aunque la ambulancia esté ocupada	2	Se invoca `trasladarALaClinica(asociadoMock)`	El método retorna sin esperar y el estado no recibe llamadas
Que, con la ambulancia libre y la simulación activa, el hilo no espere y el pedido llegue al estado	3	Un hilo invoca `trasladarALaClinica(asociadoMock)`	El hilo termina de inmediato y el estado recibe una única llamada a `SolicitudDeTraslado()`

Que se respete la exclusión mutua al interactuar dos hilos con la ambulancia	4	2 hilos invocan `trasladarALaClinica(asociado)` a la vez	Ambos hilos deben estar en `WAITING` al pedir la ambulancia. Al liberarla, exactamente uno debe terminar (es decir, adquirir el bloqueo, terminar y liberarla) nunca hay dos hilos dentro al mismo tiempo. Al terminar uno, la ambulancia debe quedar libre y por tanto el otro hilo debe tomar bloqueo de la misma. Al volver a regresar la ambulancia, ambos hilos deben haber terminado.
Que el hilo de `RetornoAutomatico` siga vivo mientras la simulación está activa	5	Se inicia un hilo con `RetornoAutomatico`, se espera 15 segundos (un tiempo mayor a los 10 segundos de intervalo del retorno automático)	El hilo con `RetornoAutomatico` debe seguir vivo
Que el hilo de `RetornoAutomatico` termine cuando la simulación no está activa	6	Se inicia un hilo con `RetornoAutomatico`, se espera 15 segundos (un tiempo mayor a los 10 segundos de intervalo del retorno automático)	El hilo con `RetornoAutomatico` debe no estar activo

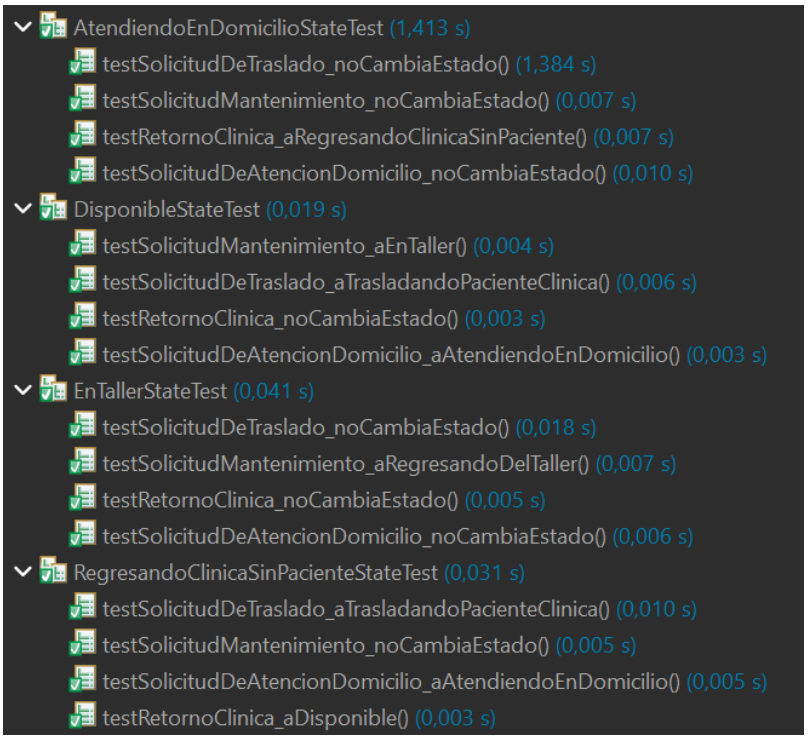
4. Errores encontrados en la ejecución

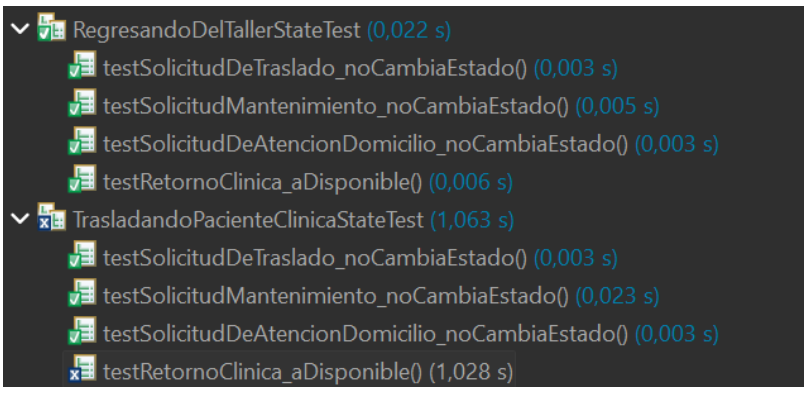
- **Test de delegación**



Se encuentra que los métodos de `Ambulancia` relacionadas al patrón State delegan correctamente a los métodos de `estadoActual`.

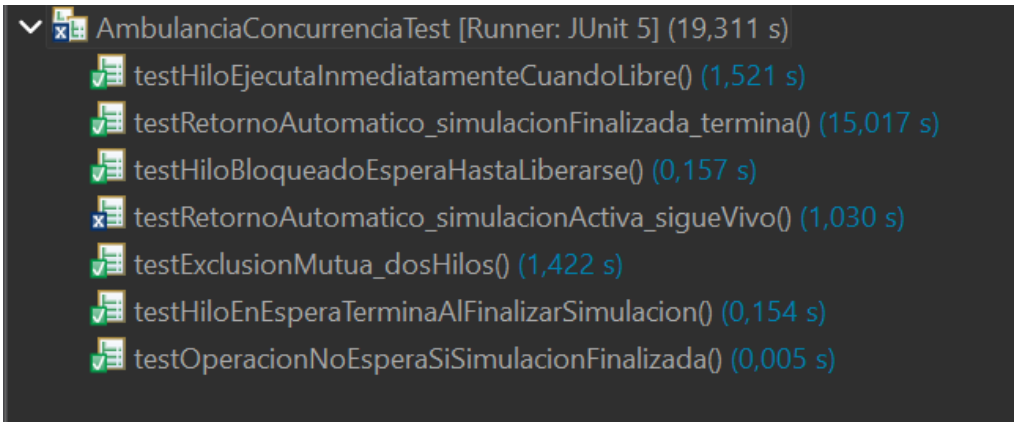
- **Test de transiciones de estado**





El único error que se encuentra es en la implementación de `TrasladandoPacienteClinicaState`, el cual en el método `RetornoClinica()` no cambia el estado de `Ambulancia` a `DisponibleState` sino a `RegresandoDelTallerState`.

- **Test de concurrencia**



El error reside en la implementación de `RetornoAutomático`, específicamente en su método `run()`. No funciona cuando la simulación está activa porque la condición del while dice `!ambulancia.isSimulacionActiva()`, en lugar de `ambulancia.isSimulacionActiva()`

Resumen de fallos detectados

Defecto	Ubicación	Pruebas que lo detectan	Resultado obtenido
Aserto incoherente con precondición en setter	AsociadoDTO.java, línea 87	testSetNumero0Valido()	Al querer asignar 0 en el número de AsociadoDTO, que por contrato es válido (número >= 0), dispara un aserto
Constante de id en base de datos	BaseDeDatos.java, línea 118	testEjecutarActualizacionModificaFilaCorrecta(), testActualizarNoAlteraAlAsociadoConId1(), testActualizarModificaElAsociadoCorrecto(), testActualizarAsociadoInexistenteNoCambiaNada().	Al siempre asignar en 1 el id de la fila a modificar en la base de datos, siempre modifica el 1er asociado, independientemente de sus parámetros
Acceso a resultado SQL invalido	DataAccessObject.java, línea 88	testGuardarCargarActualizarEliminar(), testCargarIdExistente(), testGuardarCargarEliminarDesdeCero()	Lanza SQLException al acceder a campos en una posición incorrecta de la lista resultado
Transición de estado incorrecta	TrasladandoPacienteClinicaState.java, línea 56	testRetornoClinica_aDisponible()	El método RetornoClínica() transiciona la ambulancia a RegresandoDelTallerState en lugar de DisponibleState
Condición del while incorrecta	RetornoAutomatico.java, línea 25	testRetornoAutomatico_simulacionActiva_sigueVivo()	El hilo de RetornoAutomatico evalúa !isSimulacionActiva() provocando que el hilo termine cuando la simulación está activa

Fallos estáticos hallados aclarados en la sección “4. Errores estáticos en documentación” de “Testing de la integración AsociadoDTO, BaseDeDatos y DataAccessObject”